

# CSE205:DATA STRUCTURES AND ALGORITHMS

L:3 T:0 P:2 Credits:4

**Course Outcomes:** Through this course students should be able to

CO1 :: understand the time and space complexity of programs and data-structures.

CO2 :: illustrate the importance of Linked List in context of real world problems

CO3 :: differentiate the Stack and Queue data structures for problem solving

CO4 :: use of recursion in iteration process and tree data structure

CO5 :: apply heap operations and hashing techniques for efficient data storage, retrieval, and collision resolution.

CO6 :: identify and integrating emerging data structures and algorithmic ensuring their ability to stay with technological advancements and maintain professional growth in a dynamic field

## Unit I

**Introduction** : Basic Concepts and Notations, Complexity analysis: time space and trade off, Omega Notation, Theta Notation, Big O notation, Basic Data Structures.

**Arrays** : Linear arrays: memory representation, Array operations: traversal, insertion, deletion, sorting, searching and merging and their complexity analysis.

**Sorting and Searching** : Bubble sort, Insertion sort, Selection sort, Searching: Linear Search and Binary Search

## Unit II

**Linked Lists** : Introduction, Memory representation, Allocation, Traversal, Insertion, Deletion, Header linked lists: Grounded and Circular, Two-way lists: operations on two way linked lists

## Unit III

**Stacks** : Introduction: List and Array representations, Operations on stack (traversal, push and pop), Arithmetic expressions: polish notation, evaluation and transformation of expressions.

**Queue** : Array and list representation, operations (traversal, insertion and deletion), Priority Queues, Deques

## Unit IV

**Trees** : Binary trees: introduction (complete and extended binary trees), memory representation (linked, sequential), Binary Search Tree: introduction, searching, insertion and deletion, In-order traversal, Pre-order traversal, Post-order traversal using recursion

**Recursion** : Introduction, Recursive implementation of Towers of Hanoi, Merge sort, Quick sort

## Unit V

**Heaps and Hashing** : Heaps: Introduction, Insertion, Deletion, HeapSort, Hashing introduction: hash functions, hash table, Open hashing (separate chaining), Closed hashing (open addressing): linear probing, quadratic probing and double hashing

## Unit VI

**Graphs** : Graph Traversal: BFS, DFS, Shortest path algorithm: Dijkstra's Algorithm, Bellman-Ford Algorithm, Floyd Warshall Algorithm

### List of Practicals / Experiments:

- Program to implement insertion and deletion operations in arrays
- Program to implement different searching techniques - linear and binary search
- Program to implement different sorting techniques – bubble, selection and insertion sort
- Program to implement searching, insertion and deletion operations in linked list
- Program to implement searching, insertion and deletion operations in doubly linked list
- Program to implement push and pop operations in stacks using both arrays and linked list
- Program to implement enqueue and dequeue operations in queues using both arrays and linked list
- Program to demonstrate concept of recursions with problem of tower of Hanoi
- Program to implement recursive sorting techniques - merge sort, quick sort
- Program to create and traverse a binary tree recursively
- Program to find all paths from root to leaf in a binary tree
- Program to find the Lowest Common Ancestor (LCA) of two nodes in a binary tree

- Program to implement insertion and deletion operations in BST
- Program to find the Kth smallest and Kth largest element in a Binary Search Tree
- Program to implement insertion and deletion operations in Heaps and Heap Sort
- Program to implement hash table with linear and quadratic probing
- Program to implement graph using adjacency matrix and adjacency list
- Program to implement BFS and DFS traversal on a graph
- Program to implement Dijkstra's algorithms
- Program to implement Floyd-Warshall algorithms
- Program to detect a cycle in a graph using DFS
- Program to detect and remove a loop in a linked list using Floyd's cycle detection algorithm

**Text Books:**

1. DATA STRUCTURES by SEYMOUR LIPSCHUTZ, MCGRAW HILL EDUCATION

**References:**

1. DATA STRUCTURES AND ALGORITHMS by ALFRED V. AHO, JEFFREY D. ULLMAN AND JOHN E. HOPCROFT, PEARSON